

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH & TRUYỀN THÔNG

BÁO CÁO TIẾN ĐỘ ĐỒ ÁN CHUYÊN NGÀNH

Thiết kế và Triển khai Quy trình DevSecOps
Kết hợp GitOps cho Hệ thống Microservices trên Nền tảng AWS

Giảng viên hướng dẫn: Lê Anh Tuấn

Sinh viên thực hiện: Hồ Nhật Minh

MSSV: 23520924

Mục lục

1	Tổng quan đề tài	3
1.1	Mục tiêu	3
1.2	Phạm vi công nghệ	3
1.3	Sample Application	3
2	Kiến trúc hệ thống	3
2.1	Sơ đồ tổng thể	3
2.2	Hạ tầng AWS đã triển khai	4
2.3	Mô hình 2-repo GitOps	4
3	Khối lượng đã hoàn thành	5
3.1	Giai đoạn 1: Lập kế hoạch và thiết lập	5
3.2	Giai đoạn 2: Infrastructure as Code	6
3.2.1	Terraform Backend — S3 + DynamoDB	6
3.2.2	VPC — Mạng nền tảng	6
3.2.3	EKS Cluster — Kubernetes Control Plane	7
3.2.4	Managed Node Groups	8
3.2.5	Kubernetes Addons	8
4	Kết quả kiểm thử	9
4.1	Kết nối cluster	9
4.2	Trạng thái worker nodes	9
4.3	Trạng thái tất cả pods	9
4.4	Kiểm tra trên AWS Console	9
5	Quản lý chi phí	10
5.1	Phân tích chi phí dự kiến	10
5.2	Biện pháp tiết kiệm đã triển khai	10
6	Kế hoạch tiếp theo	11
7	Rủi ro và biện pháp kiểm soát	11
8	Cấu trúc mã nguồn	11
9	Kết luận	12

Tổng quan đề tài

Mục tiêu

Đề tài hướng tới thiết kế và triển khai một quy trình DevSecOps tích hợp GitOps để tự động hóa việc build, kiểm thử bảo mật và triển khai hệ thống microservices lên nền tảng AWS. Quy trình đảm bảo:

- Mọi thay đổi mã nguồn được kiểm thử và quét bảo mật trước khi đến môi trường production.
- Việc triển khai tuân theo mô hình GitOps — toàn bộ trạng thái hệ thống được khai báo trong Git, Argo CD tự động đồng bộ lên Kubernetes.
- Bảo mật được tích hợp từ đầu (shift-left security): SAST, dependency scan, container image scan.
- Hạ tầng được quản lý 100% bằng Infrastructure as Code (Terraform).

Phạm vi công nghệ

Thành phần	Công nghệ
Hạ tầng (IaC)	Terraform > 1.5, AWS Provider > 5.0
Kubernetes	Amazon EKS v1.33, managed node groups
CI/CD	Jenkins (CI) + Argo CD (CD GitOps)
Bảo mật	SonarQube SAST, OWASP Dependency Check, Trivy
Container Registry	Amazon ECR
Giám sát	CloudWatch + Prometheus + Grafana
Sample Application	aws-containers/retail-store-sample-app (5 microservices)

Sample Application

Hệ thống mẫu là ứng dụng bán lẻ trực tuyến gồm 5 microservice:

Service	Ngôn ngữ	Framework	Backend
UI	Java 21	Spring Boot + Thymeleaf	WebFlux API Gateway
Cart	Java 21	Spring Boot	DynamoDB
Orders	Java 21	Spring Boot	DynamoDB, SQS, SNS, Secrets Manager
Catalog	Go	Gin	MySQL/MariaDB
Checkout	TypeScript	NestJS 11	Redis

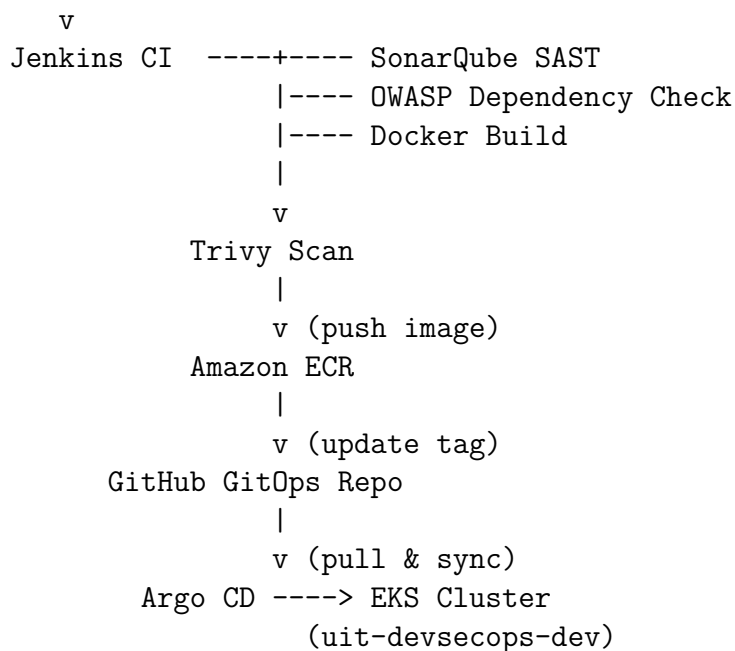
Kiến trúc hệ thống

Sơ đồ tổng thể

```

Developer
|
v
GitHub App Repo
|

```



Hạ tầng AWS đã triển khai

AWS Region: ap-southeast-1 (Singapore)

VPC (10.0.0.0/16)

Public Subnets x 3 (1 per AZ)

-> Internet Gateway (IGW)

-> Application Load Balancer (ALB, public-facing)

Private Subnets x 3 (1 per AZ)

-> EKS Worker Nodes x 2 (t3.medium)

-> NAT Gateway (single, shared)

EKS Control Plane v1.33

-> VPC-CNI, CoreDNS, kube-proxy

-> AWS Load Balancer Controller

-> Cert Manager

-> OTEL Operator (ADOT)

Supporting services:

-> KMS (secrets encryption)

-> OIDC Provider (IRSA)

-> CloudWatch Logs

S3: uit-devsecops-gitops-dev-xxxxx-tfstate (state)

DDB: uit-devsecops-gitops-dev-xxxxx-tflock (lock)

Mô hình 2-repo GitOps

Hệ thống sử dụng chiến lược tách biệt hai repository:

1. **App Repo** — chứa mã nguồn microservices, Dockerfile, Jenkinsfile. Developer push code

vào đây để kích hoạt CI pipeline.

2. **GitOps Repo** — chứa các manifest triển khai (Kustomize/Helm). CI pipeline tự động cập nhật image tag vào repo này sau khi build thành công. Argo CD theo dõi repo này và tự động đồng bộ lên EKS.

Lợi ích của mô hình này:

- Tách biệt quyền: developer chỉ cần quyền trên App Repo, không cần quyền trực tiếp lên production.
- Audit trail hoàn chỉnh: mọi thay đổi production đều có commit tương ứng trong GitOps Repo.
- Rollback = git revert: chỉ cần revert commit trong GitOps Repo, Argo CD tự động quay về trạng thái cũ.

Khối lượng đã hoàn thành

Giai đoạn 1: Lập kế hoạch và thiết lập

Hạng mục	Nội dung đã thực hiện
Cập nhật đề tài	Thống nhất tên đề tài mới: <i>Thiết kế và Triển khai Quy trình DevSecOps kết hợp GitOps cho Hệ thống Microservices trên Nền tảng AWS.</i>
So sánh mô hình cũ/mới	Đã phân tích chi tiết sự khác biệt giữa mô hình cũ (Java/Tomcat triển khai thủ công lên EC2) và mô hình mới (Microservices/EKS/GitOps). Bảng so sánh gồm 8 tiêu chí: deploy strategy, rollback, security scanning, audit trail, infrastructure management, scaling, secrets management, observability.
Chọn sample app	Đã đánh giá 3 ứng dụng mẫu do GVHD đề xuất và chọn <code>aws-containers/retail-store-sample-app</code> vì: đa dạng ngôn ngữ (Java, Go, TypeScript), kiến trúc microservices đầy đủ, hỗ trợ DynamoDB/Redis, có sẵn Dockerfile và Terraform module.
Sơ đồ kiến trúc	Đã vẽ sơ đồ tổng thể quy trình DevSecOps + GitOps bằng Mermaid, bao gồm toàn bộ luồng từ Developer → CI → Security Gates → ECR → GitOps → Argo CD → EKS.
Workspace đồ án	Đã tạo cấu trúc thư mục chuẩn gồm 10 danh mục: 00-admin, 01-docs, 02-repos, 03-infra, 04-platform, 05-ci, 06-security, 07-observability, 08-evidence, 09-scripts, 10-agent-skill. Mỗi danh mục có file README mô tả mục đích.
Clone app-repo	Đã clone toàn bộ mã nguồn sample app vào 02-repos/app-repo/, bao gồm đầy đủ Terraform modules của app (VPC, EKS, tags).

GitOps repo skeleton	Đã tạo cấu trúc thư mục cho GitOps Repo theo mô hình Kustomize: base/ + overlays/dev/, overlays/staging/, overlays/prod/ cùng với thư mục cấu hình Argo CD (argocd/applications, argocd/projects).
Tài liệu hướng dẫn	Đã viết PROJECT_ROADMAP.md (lộ trình 4 phase), HUONG_DAN_THUC_HIEN_CHI_TIET.md (hướng dẫn từng bước), và START_HERE_PHASE1.md (khởi động nhanh).
Công cụ local	Đã cài đặt và xác minh toolchain: Terraform v1.14.8, kubectl v1.34.1, Helm v4.1.3, Docker v29.2.1, AWS CLI v2.31.22. Có script tự động cài đặt (setup-tools-windows.ps1) trong 09-scripts/.

Giai đoạn 2: Infrastructure as Code

3.2.1 Terraform Backend — S3 + DynamoDB

Terraform backend là thành phần nền móng của toàn bộ kiến trúc IaC, đảm bảo:

- **State lưu tập trung trên S3** — tránh mất mát khi làm việc trên nhiều máy, hỗ trợ collaboration.
- **State locking qua DynamoDB** — ngăn chặn nhiều người cùng apply gây race condition.
- **Versioning + Encryption** — cho phép rollback state và bảo vệ dữ liệu nhạy cảm.
- **Block public access** — bucket không thể truy cập từ internet.

Resource	Chi tiết
S3 Bucket	uit-devsecops-gitops-dev-0a502713-tfstate
DynamoDB Table	uit-devsecops-gitops-dev-0a502713-tflock
Mã hóa	AES256 (server-side encryption)
Phiên bản	Enabled
Public access	Block tất cả
Terraform module	Tự viết (72 dòng), không dùng module bên ngoài

Listing 1: Cấu hình backend cho môi trường dev

```
# backend.hcl
bucket      = "uit-devsecops-gitops-dev-0a502713-tfstate"
key         = "envs/dev/terraform.tfstate"
region      = "ap-southeast-1"
dynamodb_table = "uit-devsecops-gitops-dev-0a502713-tflock"
encrypt     = true
```

3.2.2 VPC — Mạng nền tảng

VPC được tạo bằng Terraform module terraform-aws-modules/vpc/aws v5.21.0 với cấu hình tối ưu cho chi phí:

Thông số	Giá trị
VPC ID	vpc-0d37b4e9b21e95421
CIDR Block	10.0.0.0/16
Public Subnets	3 (mỗi /24, 1 mỗi Availability Zone)
Private Subnets	3 (mỗi /24, 1 mỗi AZ)
NAT Gateway	1 (single, thay vì 3 để tiết kiệm)
Internet Gateway	Đã tạo và attach
DNS Hostnames	Enabled

Các subnet được gán tag tự động cho EKS nhận diện:

- Public subnet: `kubernetes.io/role/elb = 1` — cho phép tạo Internet-facing Load Balancer.
- Private subnet: `kubernetes.io/role/internal-elb = 1` — cho phép tạo Internal Load Balancer.
- Tất cả subnet: `kubernetes.io/cluster/<name> = shared` — EKS biết subnet này thuộc cluster nào.

3.2.3 EKS Cluster — Kubernetes Control Plane

EKS cluster được tạo bằng module `terraform-aws-modules/eks/aws` ~ 19.9 với các cấu hình bảo mật:

Thông số	Giá trị
Tên cluster	uit-devsecops-dev
Phiên bản Kubernetes	v1.33.11
Control plane endpoint	Public + Private
IAM Role (cluster)	Đã tạo với trust policy cho <code>eks.amazonaws.com</code>
IAM Policies gán cho role	AmazonEKSClusterPolicy, AmazonEKSVPCResourceController
KMS Encryption	Đã bật (mã hóa Kubernetes secrets)
OIDC Provider	Đã tạo (hỗ trợ IRSA)
CloudWatch Logs	API, Audit, Authenticator → retention 90 ngày

Listing 2: Khai báo EKS cluster trong Terraform

```
module "eks_cluster" {
  source = "terraform-aws-modules/eks/aws"
  version = "~> 19.9"

  cluster_name      = var.environment_name
  cluster_version   = "1.33"
  cluster_endpoint_public_access = true

  vpc_id    = var.vpc_id
  subnet_ids = var.subnet_ids

  cluster_addons = {
    vpc-cni = {
      before_compute = true
    }
  }
}
```

```

    most_recent = true
  }
}
}

```

3.2.4 Managed Node Groups

Hai managed node group được triển khai trên hai Availability Zone khác nhau để đảm bảo high availability:

Node Group	Instance	Scaling (Min/Desired/Max)
managed-nodegroup-1	t3.medium	0 / 0 / 1
managed-nodegroup-2	t3.medium	0 / 0 / 1

- Mỗi node group nằm trong một private subnet riêng (AZ riêng).
- Instance t3.medium: 2 vCPU, 4GB RAM — đủ chạy 5 microservice của sample app.
- Hiện tại node đã được scale về 0 để tiết kiệm chi phí. Khi cần làm việc, chỉ cần thay đổi `node_min_size = 1, node_desired_size = 1` và chạy `terraform apply`.

3.2.5 Kubernetes Addons

Addon	Namespace / Số pods	Mô tả
VPC-CNI	managed, 2 pods	Quản lý địa chỉ IP cho pods thông qua Elastic Network Interface (ENI). Cho phép mỗi pod có IP riêng trong VPC.
CoreDNS	kube-system, 2 pods	DNS server nội bộ cho cluster, phân giải tên service thành IP.
kube-proxy	kube-system, 2 pods	Network proxy, quản lý Kubernetes Service (iptables/IPVS rules).
AWS Load Balancer Controller	kube-system, 2 pods	Tự động tạo Application Load Balancer (ALB) hoặc Network Load Balancer (NLB) khi tạo Kubernetes Ingress hoặc Service type LoadBalancer. Đây là thành phần quan trọng để expose service ra internet.
Cert Manager	cert-manager, 3 pods	Tự động cấp và gia hạn chứng chỉ TLS từ Let's Encrypt hoặc các CA khác. Gồm controller, cainjector và webhook.
OpenTelemetry Operator	opentelemetry-operator, 1 pod	Operator quản lý OpenTelemetry collector. Hiện collector đang ở chế độ disable, sẽ kích hoạt trong Phase 4 (Observability).

Tất cả addons được cài đặt thông qua Terraform (Helm provider cho LB Controller và Cert Manager, `aws_eks_addon` cho VPC-CNI và ADOT), đảm bảo tính nhất quán và khả năng tái tạo.

Kết quả kiểm thử

Sau khi provision hoàn tất, đã thực hiện các bước xác minh hệ thống:

Kết nối cluster

Listing 3: Xác minh kết nối đến EKS

```
$ kubectl cluster-info
Kubernetes control plane is running at
https://F358B0B27A319352351851239024C3F2.gr7.ap-southeast-1.eks.amazonaws.com
CoreDNS is running at
https://F358B0B27A319352351851239024C3F2.gr7.../api/v1/.../kube-dns:dns/proxy
```

Trạng thái worker nodes

Listing 4: Danh sách nodes — tất cả Ready

```
$ kubectl get nodes -o wide
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP
ip-10-0-10-249...	Ready	<none>	2m	v1.33.11-eks-4136f65	10.0.10.249
ip-10-0-11-155...	Ready	<none>	2m	v1.33.11-eks-4136f65	10.0.11.155

Cả hai node đều ở trạng thái Ready, chạy Amazon Linux 2023, container runtime containerd, địa chỉ IP nằm trong dải private subnet.

Trạng thái tất cả pods

Listing 5: Toàn bộ pods đều Running, không restart

```
$ kubectl get pods -A
```

NAMESPACE	NAME	READY	STATUS	RESTARTS
cert-manager	cert-manager-6c4645d66c-vn7th	1/1	Running	0
cert-manager	cert-manager-cainjector-...	1/1	Running	0
cert-manager	cert-manager-webhook-...	1/1	Running	0
kube-system	aws-load-balancer-controller-...	1/1	Running	0
kube-system	aws-load-balancer-controller-...	1/1	Running	0
kube-system	aws-node-s9glr	2/2	Running	0
kube-system	aws-node-whlpn	2/2	Running	0
kube-system	coredns-6cfd9cdf-bmsc6	1/1	Running	0
kube-system	coredns-6cfd9cdf-xbqv2	1/1	Running	0
kube-system	kube-proxy-5vcwh	1/1	Running	0
kube-system	kube-proxy-pl9vf	1/1	Running	0
opentelemetry	opentelemetry-operator-...	2/2	Running	0

Tất cả 12 pods đều ở trạng thái Running, không có pod nào bị restart — cluster hoạt động ổn định.

Kiểm tra trên AWS Console

- **EKS Dashboard:** cluster uit-devsecops-dev hiển thị trạng thái Active, 2 node groups hiển thị Active.

- **EC2 Dashboard:** 2 instances t3.medium đang chạy trong private subnet.
- **VPC Dashboard:** VPC với đầy đủ 6 subnets, Internet Gateway, NAT Gateway, route tables được cấu hình đúng.
- **CloudWatch:** Log group /aws/eks/uit-devsecops-dev/cluster đã được tạo và đang nhận log từ control plane.

Quản lý chi phí

Phân tích chi phí dự kiến

Tài nguyên	Đơn giá	Tháng (ước tính)
EKS Control Plane	\$0.10/giờ	\$72.00
2 × EC2 t3.medium (khi chạy)	\$0.0528/giờ/node	\$76.00
NAT Gateway	\$0.045/giờ	\$32.00
Tổng (full-time)		\$180.00
Tổng (node off)		\$104.00

Biện pháp tiết kiệm đã triển khai

1. **Node groups scale về 0** ngoài giờ làm việc — giảm \$76/tháng. Cấu hình đơn giản qua terraform.tfvars: node_min_size = 0, node_desired_size = 0.
2. **Single NAT Gateway** thay vì 3 (mỗi AZ một cái) — giảm chi phí NAT xuống còn 1/3.
3. **Không dùng Istio** service mesh (istio_enabled = false) — tránh overhead tài nguyên và chi phí.
4. **Không kích hoạt OpenTelemetry collector** — chỉ cài operator, chưa bật data pipeline.
5. **Dùng t3.medium** thay vì t3.large hoặc các instance lớn hơn.
6. **Terraform destroy** toàn bộ khi nghỉ dài ngày — state lưu trên S3 nên có thể tái tạo lại chính xác trong ~13 phút.

Kế hoạch tiếp theo

#	Công việc	Giai đoạn	Mức ưu tiên
1	Tạo ECR repositories cho 5 microservice	Phase 2	Cao
2	Viết Jenkinsfile: build → test → scan → push ECR	Phase 3	Cao
3	Tích hợp SonarQube SAST vào CI pipeline	Phase 3	Cao
4	Tích hợp OWASP Dependency Check	Phase 3	Trung bình
5	Tích hợp Trivy Container Image Scan	Phase 3	Trung bình
6	Viết Kustomize manifests (base + dev overlay)	Phase 4	Cao
7	Cài đặt và cấu hình Argo CD trên EKS	Phase 4	Cao
8	Tự động cập nhật image tag vào GitOps repo từ CI	Phase 4	Trung bình
9	Demo deploy toàn bộ 5 service lên EKS qua Argo CD	Phase 4	Cao
10	Demo rollback bằng git revert trên GitOps repo	Phase 4	Cao
11	Thiết lập CloudWatch + Prometheus + Grafana dashboards	Phase 4	Thấp
12	Viết runbooks và cấu hình alerts	Phase 4	Thấp
13	Tổng hợp evidence, hoàn thiện báo cáo cuối kỳ	Tổng kết	Cao

Rủi ro và biện pháp kiểm soát

Rủi ro	Mức độ	Biện pháp
IAM permissions hạn chế	Đã xử lý	IAM user uit-devsecops-gitops-deployer đã được gán quyền AdministratorAccess. Với tài khoản thật cần tuân thủ least-privilege — tài liệu tham khảo policy đã được chuẩn bị.
Chi phí AWS vượt ngân sách	Đã kiểm soát	Scale node về 0 ngoài giờ, single NAT Gateway, dùng t3.medium. Có thể terraform destroy toàn bộ khi nghỉ dài ngày.
Thời gian provision EKS lâu (~15 phút)	Thấp	Terraform tự động hóa hoàn toàn, không cần can thiệp thủ công. Có thể chạy nền trong lúc làm việc khác.
Phiên bản module không tương thích	Thấp	Tất cả module được pin version cụ thể (~ 19.9, 5.21.0), không dùng latest.
Quên tắt tài nguyên gây lãng phí	Đã kiểm soát	File terraform.tfvars đã được cấu hình sẵn cho chế độ tiết kiệm (node = 0). Khi cần làm việc tiếp chỉ cần sửa lại node_desired_size = 1 và terraform apply.

Cấu trúc mã nguồn

do-an-devsecops-gitops/

00-admin/	Quan ly du an (meeting notes, decision log)
01-docs/	Tai lieu (report, slides, thesis plan)
report/	Bao cao tien do
02-repos/	Ma nguon ung dung & GitOps manifests
app-repo/	Sample app (aws-containers/retail-store-sample-app)
src/	5 microservices (ui, cart, orders, catalog, checkout)
terraform/	Terraform modules cua app (VPC, EKS, tags)
gitops-repo/	GitOps deployment manifests (Kustomize)
apps/retail-store/	
base/	
overlays/{dev,staging,prod}/	
03-infra/terraform/	Ha tang cua do an (Terraform)
backend/	S3 + DynamoDB state backend
envs/dev/	Moi truong dev (VPC + EKS)
04-platform/	Kubernetes platform add-ons (Argo CD)
05-ci/	Jenkins CI pipeline (Jenkinsfile)
06-security/	Security artifacts (policies, SBOM, scan reports)
07-observability/	Monitoring (alerts, dashboards, runbooks)
08-evidence/	Bang chung cho tung phase
09-scripts/	Script ho tro (setup toolchain, switch profile)
10-agent-skill/	AI assistant skill definition

Kết luận

Đồ án đã hoàn thành giai đoạn lập kế hoạch (Phase 1) và đang tiến hành giai đoạn hạ tầng (Phase 2). Các kết quả chính đạt được đến thời điểm hiện tại:

1. **Kiến trúc rõ ràng:** Mô hình DevSecOps + GitOps đã được thiết kế chi tiết, bao gồm sơ đồ luồng dữ liệu, danh sách AWS services, và kế hoạch triển khai theo 4 phase.
2. **Hạ tầng sẵn sàng:** VPC, EKS cluster v1.33, và toàn bộ addons (Load Balancer Controller, Cert Manager, OTEL Operator) đã được provision thành công bằng Terraform, kiểm thử và xác minh hoạt động ổn định.
3. **Chi phí được kiểm soát:** Đã triển khai các biện pháp tiết kiệm (scale node về 0, single NAT, instance t3.medium), tổng chi phí tối đa \$104/tháng, có thể giảm về \$0 khi terraform destroy.
4. **Tài liệu đầy đủ:** Workspace được tổ chức khoa học, có sẵn tài liệu roadmap, hướng dẫn thực hiện, và script tự động hóa.

Các bước tiếp theo sẽ tập trung vào việc xây dựng CI pipeline với security gates (Jenkins + SonarQube + OWASP + Trivy), tạo Kustomize manifests, và triển khai CD qua Argo CD.